

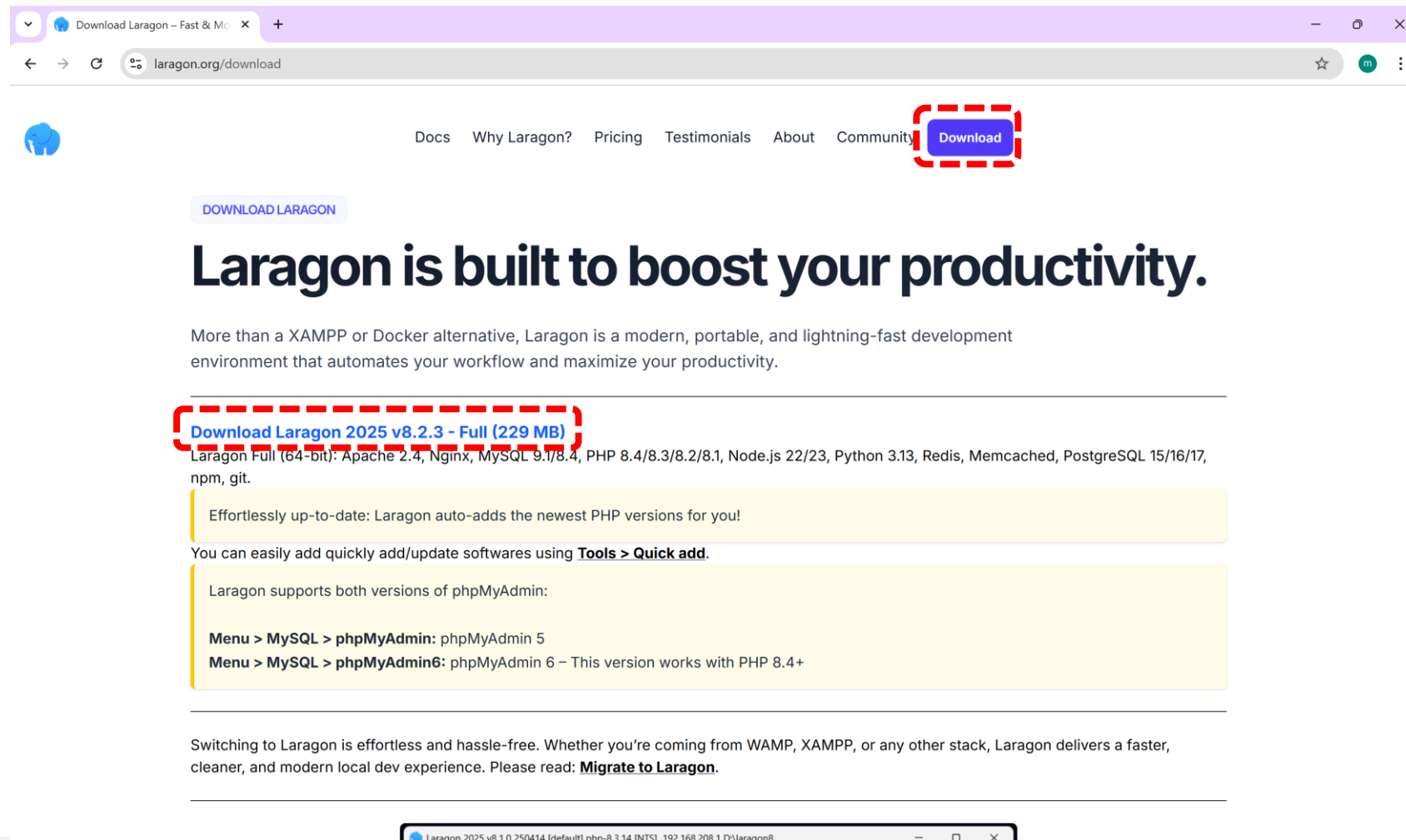


การเชื่อมต่อฐานข้อมูล MySQL

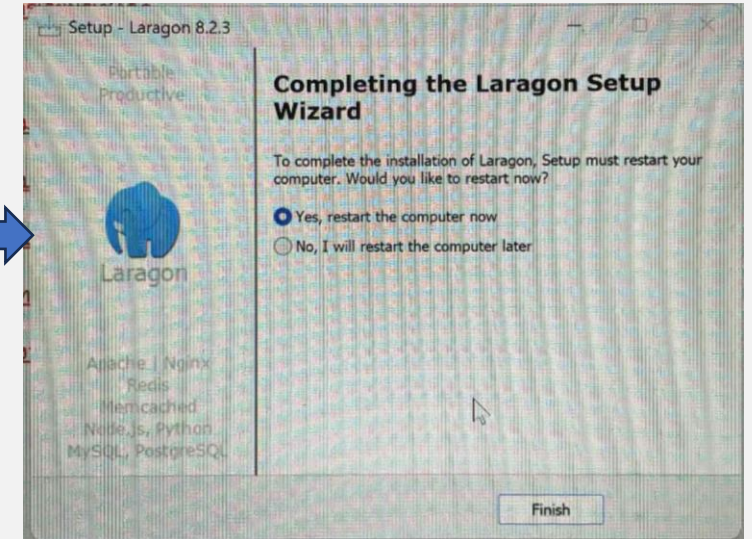
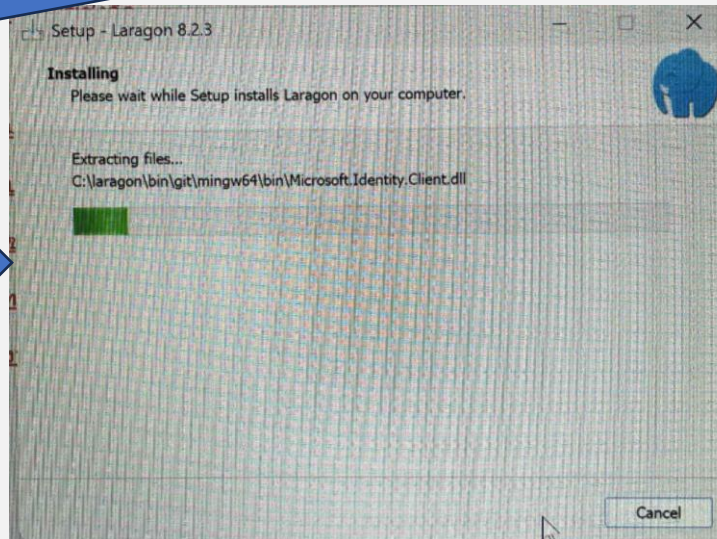
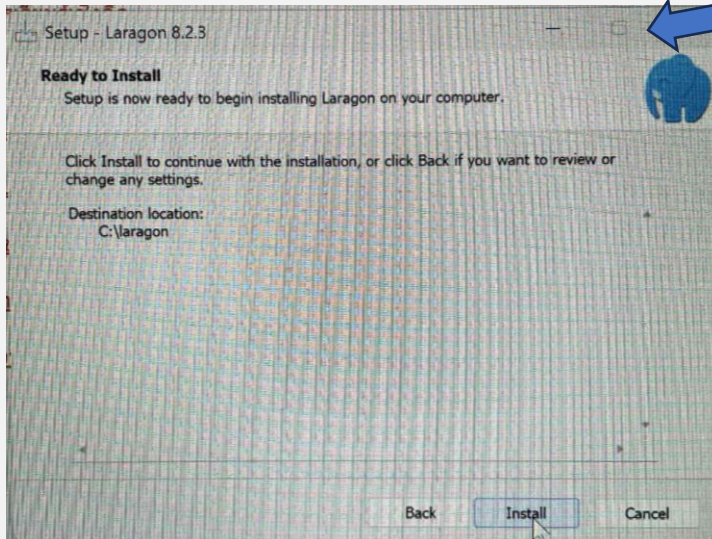
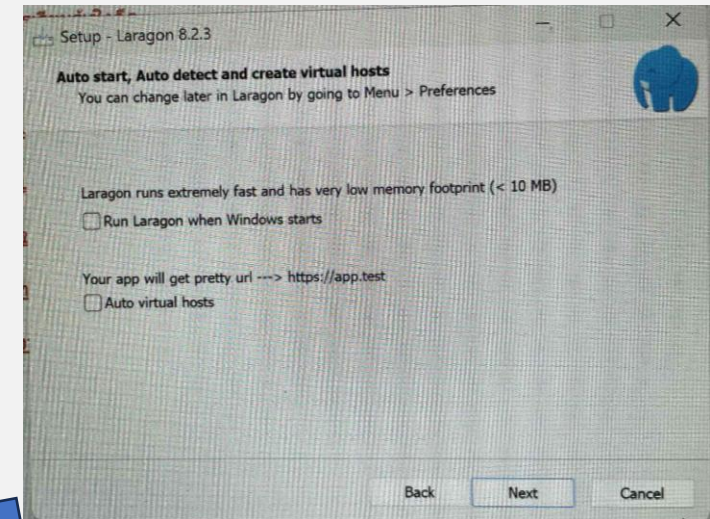
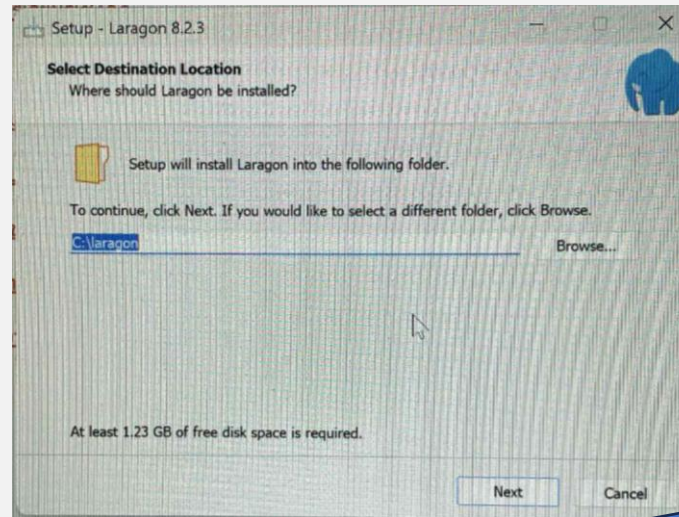
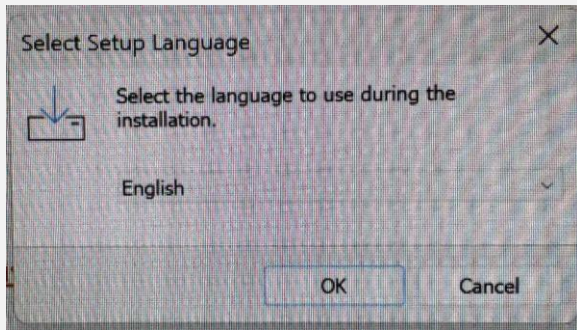


Laragon

- <https://laragon.org/download>

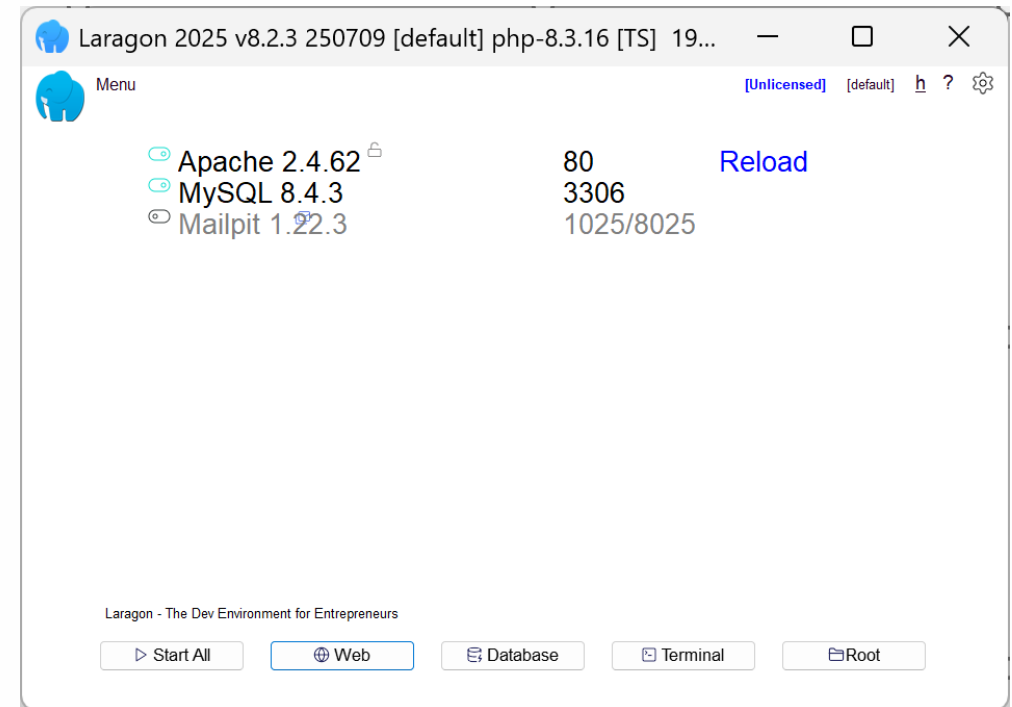
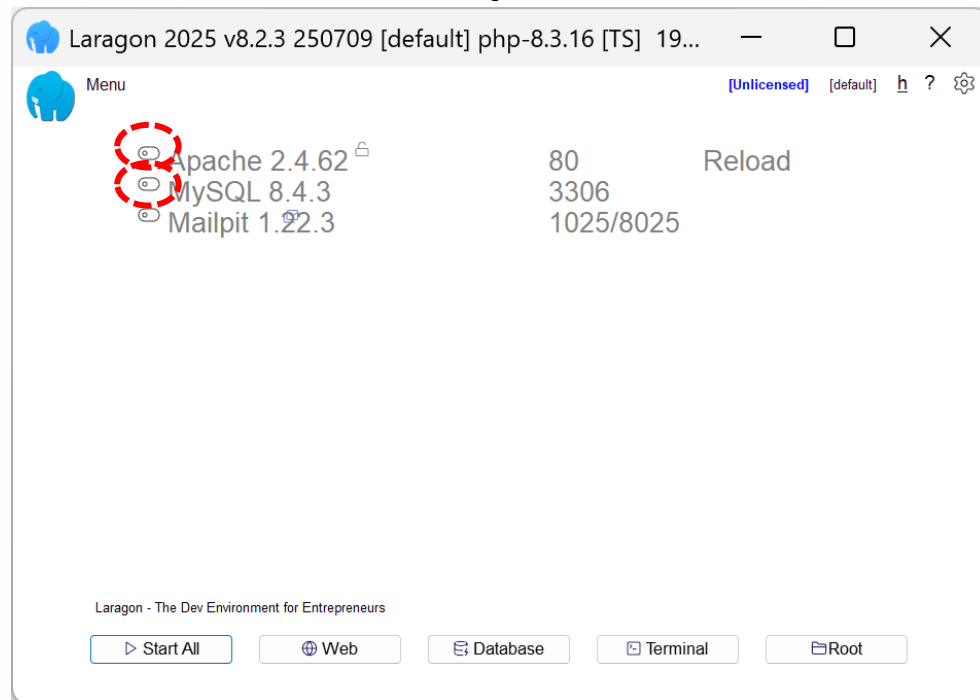


Setup-Laragon

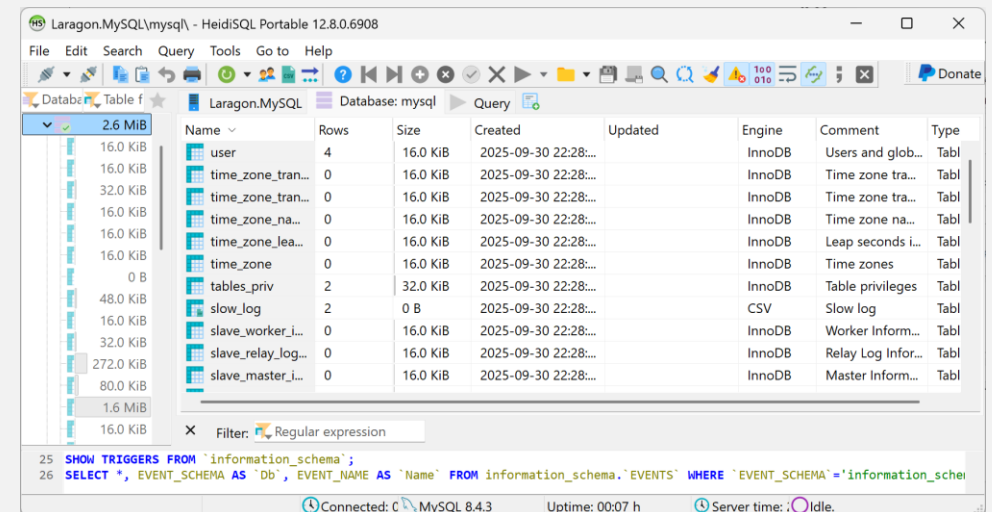
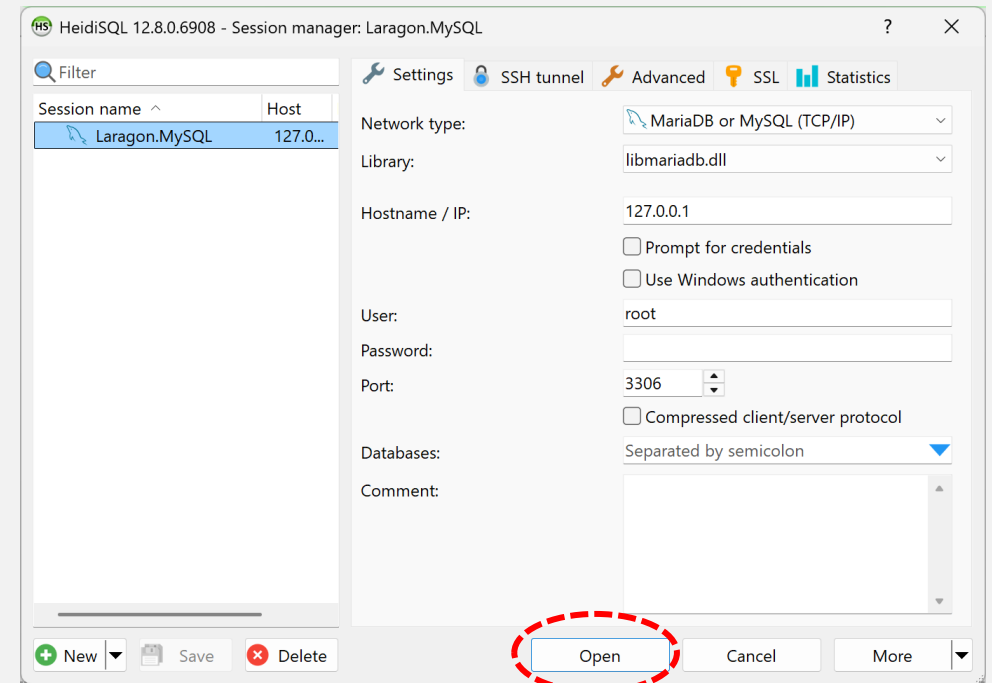
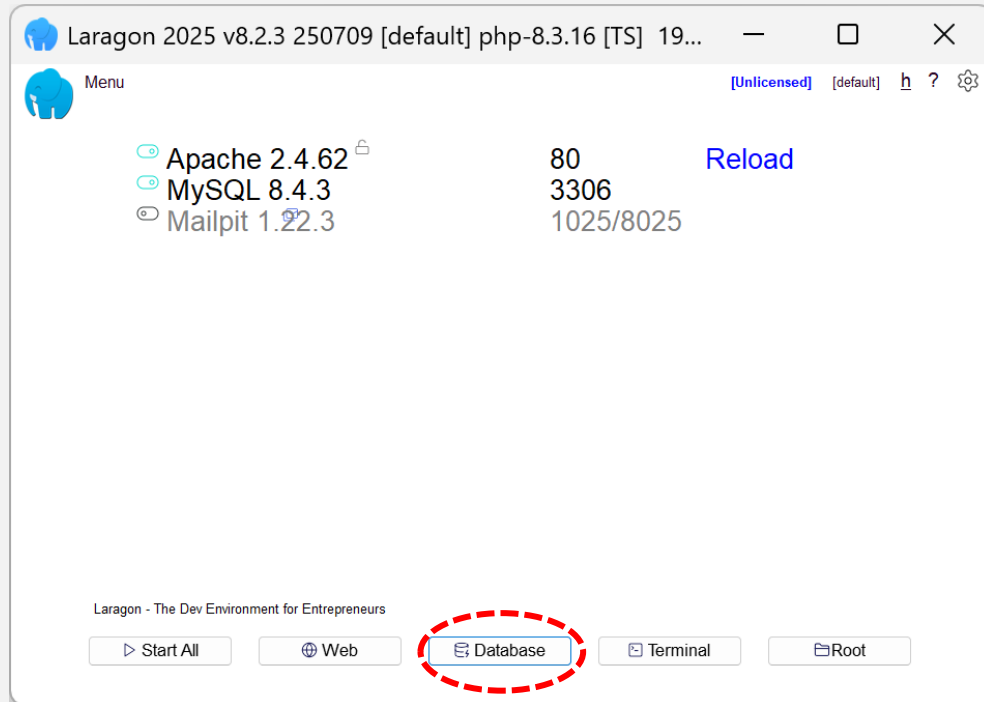


เริ่มต้นใช้งาน Laragon

- เปิดเซิร์ฟเวอร์ Apache (ที่เรียกอย่างเป็นทางการว่า Apache HTTP Server หรือ httpd)
- เปิด ระบบจัดการฐานข้อมูล MySQL



เปิด Database



ใช้ phpMyAdmin

- คลิกขวา → Tools → Quick add → phpMyAdmin

The image shows the Laragon 2025 v8.2.3 interface. The 'Tools' menu is open, and the 'Quick add' submenu is visible. The 'phpMyAdmin' option is highlighted in the submenu. A blue arrow points from the 'phpMyAdmin' option to the phpMyAdmin login page.

The phpMyAdmin login page is displayed, showing the 'Welcome to phpMyAdmin' message and the 'Log in' button. The login form includes fields for 'Username' (root) and 'Password'.

สร้างฐานข้อมูล

- **utf8_unicode_ci** และ **utf8_general_ci** แตกต่างกันคือ รูปแบบการจัดเรียงข้อมูล (Collation) ของฐานข้อมูล
- **general** คือกติกาการเปรียบเทียบตัวอักษร (collation rule) general จะไม่เป๊ะมาก แต่เร็ว (ไม่ได้แยกตัวอักษรบางแบบละเอียด เช่น ä กับ a อาจถือว่าเท่ากัน)
- **unicode** Collation ที่เปรียบเทียบตัวอักษรตามกฎ Unicode แม่นกว่า, ตามมาตรฐาน Unicode, รองรับภาษายุโรป เอเชีย อาหรับ ฯลฯ ได้ถูกต้องกว่า เหมาะกับระบบที่ต้องการความถูกต้องในการเปรียบเทียบ/จัดเรียงข้อความ เช่น ระบบที่รองรับหลายภาษา
- **ci (Case Insensitive)**
 - ไม่สนใจพิมพ์เล็ก/ใหญ่ เวลาเรียงลำดับ $\rightarrow$ A และ a ถือว่าเท่ากัน
 - ผลการเรียงอาจจะเอา A และ a ไปรวมกัน
- **cs (Case Sensitive)**
 - สนใจพิมพ์เล็ก/ใหญ่ เวลาเรียงลำดับ $\rightarrow$ $A \neq a$
 - ผลการเรียงจะจัดกลุ่มแยกกัน เช่น อักษรพิมพ์ใหญ่มาก่อน แล้วตามด้วยพิมพ์เล็ก
- **utf8mb4 (ใหม่ แนะนำ)**
 - คือ character set แบบ UTF-8 แต่เก๋กว่า
 - เก็บตัวอักษรได้สูงสุด 3 ไบต์ต่อ 1 ตัวอักษร
 - รองรับ Unicode ได้ส่วนใหญ่ เช่น ภาษาไทย ภาษาอังกฤษ จีน ญี่ปุ่น เกาหลี
 - แต่ไม่รองรับอีโมจิ (Emoji) หรืออักขระที่ต้องใช้ 4 ไบต์

1. เลือก New

2. กำหนดชื่อฐานข้อมูล

3. เลือกรูปแบบ

4. กด Create

Database	Collation	Primary replication	Action
☐ information_schema	utf8mb3_general_ci	✓ Replicated	Check privilege
☐ mysql	utf8mb4_0900_ai_ci	✓ Replicated	Check privilege
☐ performance_schema	utf8mb3_general_ci	✓ Replicated	Check privilege
☐ sys	utf8mb3_general_ci	✓ Replicated	Check privilege

CHARACTER SET

- UTF-8 คือการเข้ารหัสชุดอักขระที่ใช้ชุดข้อมูล 1 ถึง 4 byte เพื่อแทนตัวอักษรเกือบทั่วโลก
- tis-620 ของภาษาไทยเป็นที่ยอมรับกันในมาตรฐานสากล

การตั้งชื่อฐานข้อมูล

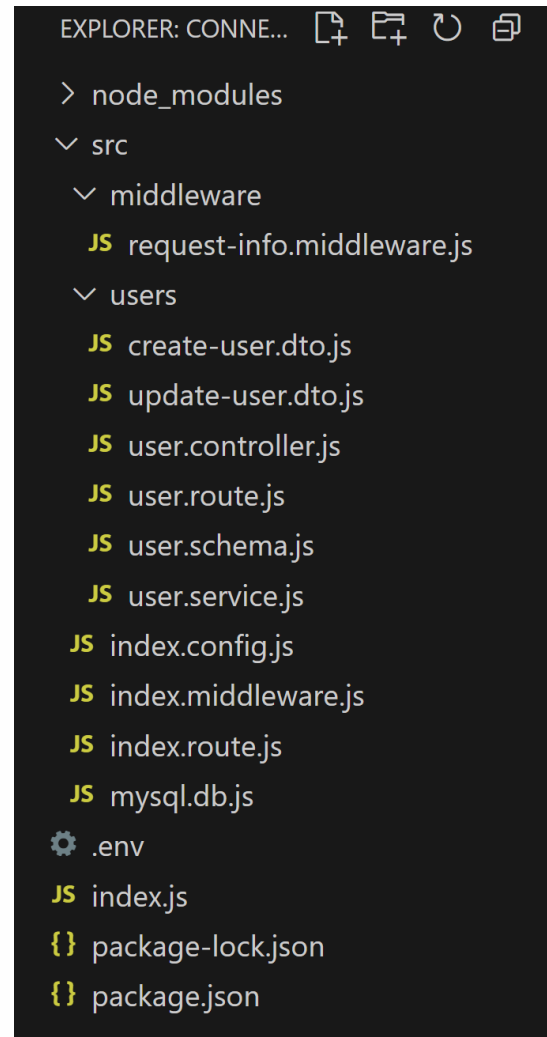
- 1. ตั้งชื่อให้สื่อความหมาย
- 2. ใช้ตัวอักษรเล็ก (Lowercase) เพื่อป้องกันปัญหาเรื่องความแตกต่างระหว่างตัวพิมพ์เล็กและตัวพิมพ์ใหญ่ในบางระบบ
- 3. หลีกเลี่ยงการใช้ช่องว่าง (Spaces) ควรใช้ _ หรือ camelCase เช่น inventory_management, inventoryManagement (แต่อาจไม่แนะนำใน SQL เนื่องจากความนิยมใช้ _ มากกว่า)
- 4. หลีกเลี่ยงการใช้คำสงวน (Reserved Keywords) เช่น ไม่ควรตั้งชื่อว่า database หรือ select
- 5. ไม่ควรตั้งชื่อยาวเกินไป ควรมีความยาวที่เหมาะสม สั้นกระชับ แต่ยังคงความหมาย
- 6. ใช้รูปแบบการตั้งชื่อแบบมีมาตรฐาน (Consistent Naming Convention) เลือกรูปแบบการตั้งชื่อและใช้ให้เหมือนกันทุกที่
- 7. ใส่คำนำหน้าหรือคำต่อท้าย (Prefix/Suffix) หากจำเป็น หากต้องการแยกประเภทของฐานข้อมูล ให้ใส่ Prefix หรือ Suffix เพื่อความชัดเจน เช่น dev_ecommerce_db (สำหรับฐานข้อมูล Development), prod_hr_management (สำหรับ Production)
- 8. หลีกเลี่ยงอักขระพิเศษ (Special Characters) เช่น @, #, \$, ! เพราะอาจทำให้เกิดปัญหาในบางระบบ
- 9. เว้นว่างการใช้ตัวเลขที่ไม่จำเป็น หากจำเป็นต้องใช้ตัวเลข ควรใช้ในบริบทที่เข้าใจได้ เช่น เวอร์ชัน project_v1

การติดตั้งไลบรารี

9

- สร้างไฟล์ package.json ด้วยคำสั่ง `npm init --y`
- ติดตั้ง Express.js ได้ผ่านคำสั่ง `npm install express`
- ติดตั้ง Nodemon ช่วย Restart node server ให้โดยอัตโนมัติโดยใช้คำสั่ง `npm install nodemon --save-dev`
- ติดตั้ง CORS packages ด้วยคำสั่ง `npm install cors`
- ติดตั้ง Joi ผ่าน: `npm install joi` สำหรับการตรวจสอบและตรวจสอบความถูกต้องของข้อมูลใน Node.js ซึ่งมักใช้ในการตรวจสอบข้อมูลที่รับมาจากการร้องขอ (requests) หรือข้อมูลที่ใช้กรอกเข้ามา เพื่อให้แน่ใจว่าข้อมูลมีรูปแบบและค่าที่ถูกต้องตามที่ต้องการ
- ติดตั้ง createValidator ใช้สร้างตัวตรวจสอบ (validator) ที่สามารถใช้ในการตรวจสอบข้อมูลที่ได้รับจาก request ผ่าน: `npm install express joi express-joi-validation`
- ติดตั้ง dotenv ด้วยคำสั่ง `npm install dotenv dotenv` เป็นแพ็คเกจที่ช่วยให้คุณสามารถโหลดค่าคอนฟิก (configuration) หรือ ตัวแปรสภาพแวดล้อม (Environment Variables) จากไฟล์ .env เข้ามาใช้งานในโปรเจกต์ Node.js ได้อย่างง่ายดาย โดยปกติจะใช้เพื่อเก็บข้อมูลที่สำคัญและควรปกปิด เช่น: คีย์ API (API keys), URL ของฐานข้อมูล (Database URLs), พอร์ตที่แอปพลิเคชันทำงาน, ข้อมูลที่ต้องการป้องกันไม่ให้เผยแพร่ในโค้ด,
- ติดตั้ง mysql: `npm install mysql2`
- เพิ่ม "type": "module" ถัดจาก "main": "index.js" ใน package.json (เป็นการเขียนแบบ ESM (ECMAScript Modules)) เพื่อใช้คำสั่ง import
- สามารถใช้งาน Web Server ของได้ด้วยการใช้คำสั่ง `npx nodemon index.js` หลังจากที่เรารันโปรแกรมแล้ว Cursor บน Command line จะค้างอยู่แบบนั้นตลอดไป นั่นเป็นเพราะว่าเว็บเซิร์ฟเวอร์กำลังรอรับคำร้องขออยู่แบบ Asynchronous และมันจะทำงานตลอดไป จนกว่าจะมีการกดปุ่ม CTRL+C เพื่อออกจากโปรแกรม
- ใส่ URL ใน Web Browser `http://localhost:3000/` (ใช้ตามที่กำหนด port)

- สร้างไฟล์ index.js ขึ้นมาตรง Path เดียวกันกับ package.json
- สร้างไฟล์ .env ขึ้นมาตรง Path เดียวกันกับ package.json ใช้เก็บค่าต่างๆ เช่น API keys, รหัสผ่าน, พอร์ต, หรือการตั้งค่าอื่นๆ ที่ไม่ต้องการให้เผยแพร่ในโค้ดลงในไฟล์
- สร้างโฟลเดอร์ src ภายในประกอบด้วย
 - ไฟล์ mysql.db.js
 - โฟลเดอร์ users
 - user.route.js
 - user.schema.js
 - user.service.js
 - user.controller.js
 - create-user.dto.js
 - update-user.dto.js
 - ไฟล์ index.route.js
 - ไฟล์ index.middleware.js
 - ไฟล์ index.config.js
 - โฟลเดอร์ middleware
 - request-info.middleware.js



กำหนดโครงสร้าง
การจัดเก็บไฟล์

สร้างไฟล์ .env

- .env คืออะไร
 - เป็นไฟล์ Environment Variables
 - ใช้เก็บ ค่าการตั้งค่า (config) ของโปรเจกต์ เช่น
 - Database host/user/password
 - API keys
 - Port ของ server
- ความปลอดภัย → ไม่ต้องเขียน password หรือ secret key ไว้ในโค้ด
- ทำงานร่วมกันง่าย → คนละเครื่องแค่แก้ไฟล์ .env ของตัวเอง ไม่ต้องแก้โค้ด
- อย่า push .env เข้า GitHub ให้ใส่ใน .gitignore แทน

.env

```
PORT = 3500
```

สร้างไฟล์ index.js

- สร้างไฟล์ index.js เป็นไฟล์จุดเริ่มต้นของแอปพลิเคชัน ซึ่งเป็นไฟล์หลักที่เริ่มต้นการทำงานของเซิร์ฟเวอร์หรือแอปพลิเคชัน
- สร้างไฟล์ package.json ด้วยคำสั่ง `npm init --y`
- ใช้ `import express from 'express'` และอย่าลืมเพิ่ม "type": "module" ถัดจาก "main": "index.js" ใน package.json (เป็นการเขียนแบบ ESM (ECMAScript Modules))
- ติดตั้ง Express.js ได้ผ่านคำสั่ง `npm install express`
- ติดตั้ง dotenv ผ่าน: `npm install dotenv`
- ติดตั้ง Nodemon ช่วย Restart node server ให้โดยอัตโนมัติ โดยใช้คำสั่ง `npm install nodemon`
- สามารถใช้งาน Web Server ของได้ด้วยการใช้คำสั่ง `npx nodemon index.js` หลังจากที่เรารันโปรแกรมแล้ว Cursor บน Command line จะค้างอยู่แบบนั้นตลอดไป นั่นเป็นเพราะว่าเว็บเซิร์ฟเวอร์กำลังรอรับคำร้องขออยู่แบบ Asynchronous และมันจะทำงานตลอดไปจนกว่าจะมีการกดปุ่ม CTRL+C เพื่อออกจากโปรแกรม
- ใส่ URL ใน Web Browser `http://localhost:3000/` (ใช้ตามที่กำหนด port)

index.js

```
import express from 'express'
import dotenv from 'dotenv'
const app = express()

dotenv.config();

app.get('/', (req, res) => {
  res.status(200).send("Hello test!!!")
})

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`http://localhost:${PORT}`)
})
```

- ติดตั้ง CORS packages ด้วยคำสั่ง `npm install cors`
- คำสั่ง `IndexMiddleware.use(express.urlencoded({ extended: true })))`
 - ใช้สำหรับจัดการข้อมูลใน Content-Type: application/x-www-form-urlencoded ซึ่งเป็นรูปแบบที่ส่งข้อมูลจากแบบฟอร์ม (HTML forms)
 - ถ้า `extended: true`, ส่งฟอร์มที่มีข้อมูล `key1[subkey]=value` จะถูกแปลงเป็น: `{ key1: { subkey: 'value' } }`
- คำสั่ง `IndexMiddleware.use(express.json())`
 - ใช้สำหรับจัดการข้อมูลใน Content-Type: application/json ซึ่งเป็นรูปแบบ JSON ที่นิยมใช้ใน API
 - Middleware นี้ช่วยให้ Express.js สามารถแปลงข้อมูลใน request body ให้กลายเป็น JavaScript object ซึ่งสามารถเข้าถึงได้ผ่าน `req.body`
- คำสั่ง `IndexMiddleware.use(cors())`
 - ใช้สำหรับเปิดใช้งาน CORS (Cross-Origin Resource Sharing) ในเซิร์ฟเวอร์ Node.js/Express.js เพื่ออนุญาตให้แอปพลิเคชันจากโดเมนหรือ origin อื่น ๆ สามารถเข้าถึง API ได้
 - CORS (Cross-Origin Resource Sharing) เป็นกลไกที่ช่วยให้เบราว์เซอร์อนุญาตหรือปฏิเสธการเรียกใช้ API ที่มาจากโดเมน (origin) ต่างกัน เช่น:
 - Origin ของ frontend: <http://example.com>
 - Origin ของ backend (API): <http://api.example.com>

สร้างไฟล์ index.middleware.js

index.middleware.js

```
import express from 'express'
import cors from 'cors'

const IndexMiddleware = express()
IndexMiddleware.use(cors())
IndexMiddleware.use(express.urlencoded({extended:true}))
IndexMiddleware.use(express.json())

export default IndexMiddleware
```

ปรับปรุงไฟล์ index.js

- เพิ่มการเรียกใช้งาน ไฟล์
index.middleware.js

```
import express from 'express'
import dotenv from 'dotenv'
import IndexMiddleware from './src/index.middleware.js'
const app = express()
app.use(IndexMiddleware)
dotenv.config();

app.get('/', (req, res) => {
  res.status(200).send("Hello test!!!")
})

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`http://localhost:${PORT}`)
})
```

สร้าง request-info.middleware.js

- สร้าง middleware เพื่อ log ข้อมูลของ request ที่ร้องขอเข้ามาทุกครั้ง จะแสดง url method และวันที่ เวลา ออกมา

request-info.middleware.js

```
const requestInfoMiddleware = function (req, res, next) {  
  console.log(`URL: ${req.originalUrl} Method: ${req.method} Date:${new Date()}`)  
  
  next()  
}  
  
export default requestInfoMiddleware
```

ไฟล์ index.middleware.js

- เพิ่มการเรียกใช้ที่ request-info.middleware.js ได้ เมื่อมี request เข้ามาจะแสดงข้อมูล

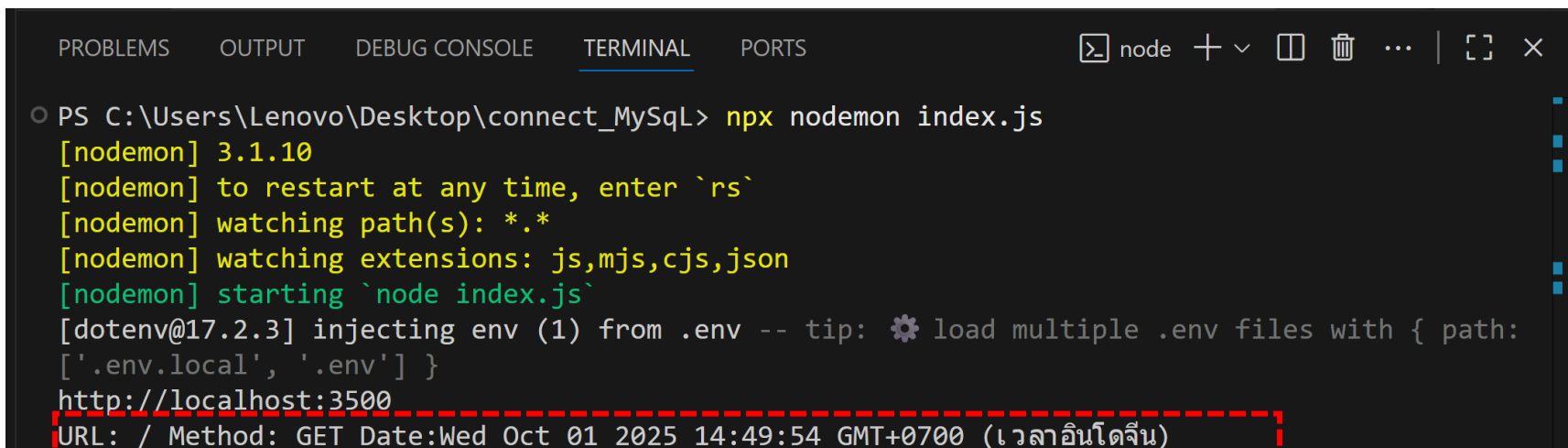
index.middleware.js

```
import express from 'express'
import cors from 'cors'
import requestInfoMiddleware from './middleware/request-info.middleware.js'

const IndexMiddleware = express()

IndexMiddleware.use(cors())
IndexMiddleware.use(express.urlencoded({extended:true}))
IndexMiddleware.use(express.json())
IndexMiddleware.use(requestInfoMiddleware)

export default IndexMiddleware
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Lenovo\Desktop\connect_MySql> npx nodemon index.js
[nodemon] 3.1.10
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
[dotenv@17.2.3] injecting env (1) from .env -- tip: ⚙️ load multiple .env files with { path:
['.env.local', '.env'] }
http://localhost:3500
URL: / Method: GET Date: Wed Oct 01 2025 14:49:54 GMT+0700 (เวลาอินโดจีน)
```

การดูชื่อ host และ user และ password ใน XAMPP

localhost/phpmyadmin/index.php?route=/server/privileges&viewing_mode=server

Server: 127.0.0.1

Databases SQL Status **User accounts** Export Import Settings Replication Variables Charsets Engines Plugins

User accounts overview

User name	Host name	Password	Global privileges	Grant	Action
Any	%	No	USAGE	No	Edit privileges Export Lock
pma	localhost	No	USAGE	No	Edit privileges Export Lock
root	127.0.0.1	No	ALL PRIVILEGES	Yes	Edit privileges Export Lock
root	::1	No	ALL PRIVILEGES	Yes	Edit privileges Export Lock
root	localhost	No	ALL PRIVILEGES	Yes	Edit privileges Export Lock

Check all With selected Export

New

Add user account

Remove selected user accounts

Revoke all active privileges from the users and delete them afterwards.

☐ Drop the databases that have the same names as the users.

Go

Note: phpMyAdmin gets the users' privileges directly from MySQL's privilege tables. The content of these tables may differ from the privileges the server uses, if they have been changed manually. In this case, you should [reload the privileges](#) before you continue.

Console

ชื่อ host และ user และ password

ปรับปรุงไฟล์ .env

- เพิ่มข้อมูลเกี่ยวกับ Database

```
PORT = 3500
```

.env

```
MYSQL_HOST=localhost #หรือชื่อโฮสต์ของ MySQL server  
MYSQL_USER=root #ชื่อผู้ใช้ MySQL  
MYSQL_PASSWORD= #รหัสผ่าน MySQL  
MYSQL_DB_NAME=new_db_test #ชื่อฐานข้อมูล MySQL ที่ต้องการเชื่อมต่อ  
MYSQL_PORT=3306 #หมายเลขพอร์ตที่ MySQL เปิดให้เชื่อมต่อ
```

- ติดตั้ง mysql: `npm install mysql2`
- ติดตั้ง dotenv ด้วยคำสั่ง `npm install dotenv`
- ปกติถ้าใช้ `mysql.createConnection()` → จะได้ connection เดียว ไปที่ MySQL ถ้าเว็บมีคนเข้ามาพร้อม ๆ กันเยอะ ๆ → connection นี้จะทำงานไม่ทัน, หลุด, หรือ error ได้
- แต่ถ้าใช้ `mysql.createPool()` → จะสร้าง “สระน้ำของ connection” เอาไว้ (มีหลาย ๆ connection รออยู่) เวลาที่มี request เข้ามา → Node.js จะหยิบ connection จาก pool ไปใช้ พอใช้เสร็จ → คืน connection กลับเข้ามาใน pool ทำให้ระบบรองรับการเชื่อมต่อพร้อมกันได้มากขึ้น, เสถียรกว่า และเร็วกว่า
 - `waitForConnections: true` → ถ้า connection เต็ม ให้รอแทนที่จะ error ทันที
 - `connectionLimit: 10` → ให้ pool สร้าง connection ได้สูงสุด 10 อัน
 - `queueLimit: 0` → ถ้า connection เต็ม จะให้ queue คำสั่งรอ (0 = ไม่จำกัด)
- ฟังก์ชัน `doQuery` ใช้ในการรันคำสั่ง SQL ผ่าน MySQL database ใน Node.js โดยใช้ Promises เพื่อจัดการกับการดำเนินการแบบอะซิงโครนัส (asynchronous) ในการสั่ง query

สร้างไฟล์ `mysql.db.js` (มีต่อ)

mysql.db.js

```
import mysql from 'mysql2';
import dotenv from 'dotenv';

dotenv.config();

//ใช้ Connection Pool
const pool = mysql.createPool({
  host      : process.env.MYSQL_HOST,
  user      : process.env.MYSQL_USER,
  password  : process.env.MYSQL_PASSWORD,
  database  : process.env.MYSQL_DB_NAME,
  waitForConnections: true, // ถ้า connection เต็ม ให้รอแทนที่จะ error ทันที
  connectionLimit: 10,    // จำนวน connection สูงสุด
  queueLimit: 0           // ถ้า 0 = ไม่จำกัดจำนวนคำสั่งที่รอ
});
```

- `pool.query()` ของ `mysql2` เป็นแบบ `callback` → เราสร้าง wrapper ให้กลายเป็น `Promise`
- ฟังก์ชัน `doQuery` ใช้ในการรันคำสั่ง SQL ผ่าน MySQL database ใน Node.js โดยใช้ `Promises` เพื่อจัดการกับการดำเนินการแบบอะซิงโครนัส (asynchronous) ในการส่ง query
 - ฟังก์ชันนี้รับ 2 argument
 - `sql` = string ของคำสั่ง SQL (เช่น "SELECT * FROM users WHERE id = ?")
 - `params` = array ของค่าที่จะใช้แทน ? ใน SQL (default = [])
 - `return new Promise((resolve, reject) => { ... })`
 - ฟังก์ชัน `doQuery` จะคืนค่าเป็น `Promise`
 - ข้อดี: สามารถใช้กับ `async/await` ได้
 - `resolve()` = คืนค่าถ้าสำเร็จ
 - `reject()` = คืน error ถ้ามีปัญหา
 - `pool.query(sql, params, (err, results) => { ... })`
 - ใช้ connection pool ไป query ฐานข้อมูล
 - `sql` → คือคำสั่ง SQL ที่ส่งไป เช่น "SELECT * FROM users WHERE id = ?"
 - `params` → array ของค่าที่แทน ? เช่น [5]
 - `callback (err, results)` → รับ error หรือผลลัพธ์จาก MySQL
 - `if (err) return reject(err);`
 - ถ้ามี error เกิดขึ้น → ส่ง error ออกไปให้ `Promise` → ฝั่งที่เรียกต้อง `catch`
 - `resolve(results);`
 - ถ้า query สำเร็จ → ส่งผลลัพธ์ (results) กลับไป

สร้างไฟล์ `mysql.db.js` (มีต่อ)

`mysql.db.js`

```
// wrapper query (Promise-based)
export const doQuery = (sql, params = []) => {
  return new Promise((resolve, reject) => {
    pool.query(sql, params, (err, results) => {
      if (err) return reject(err);
      resolve(results);
    });
  });
};
```

- threadId = หมายเลขประจำตัวของ connection ที่ MySQL server สร้างไว้ให้
- pool.getConnection((err, connection) => { ... })
 - ขอ 1 connection ออกจาก pool (ซึ่งมี connection หลายตัวอยู่แล้ว)
 - ใช้ callback รับค่า:
 - err = ถ้ามี error ตอนเชื่อมต่อ MySQL
 - connection = object ที่แทน connection จริง ๆ

สร้างไฟล์ mysql.db.js (มีต่อ)

mysql.db.js

```
// ทดสอบการเชื่อมต่อ
pool.getConnection((err, connection) => {
  if (err) {
    console.error(`Cannot connect MySQL: ${err.message}`);
    return;
  }
  console.log(`Connected MySQL as id: ${connection.threadId}`);
  connection.release(); // คืน connection ให้ pool
});

export default pool;
```

สร้างไฟล์ index.config.js ในโฟลเดอร์ src

- เป็นไฟล์จัดการการตั้งค่าการเชื่อมต่อฐานข้อมูลและการตั้งค่าอื่น ๆ สำหรับแอปพลิเคชัน

```
import express from 'express'  
import './mysql.db.js'  
const IndexConfig = express()  
export default IndexConfig
```

index.config.js

ไฟล์ index.js

- เพิ่มการเรียกใช้ไฟล์ index.config.js ลงในไฟล์ index.js
- ถ้าเชื่อต่อฐานข้อมูลสำเร็จจะเป็นดังภาพ

index.js

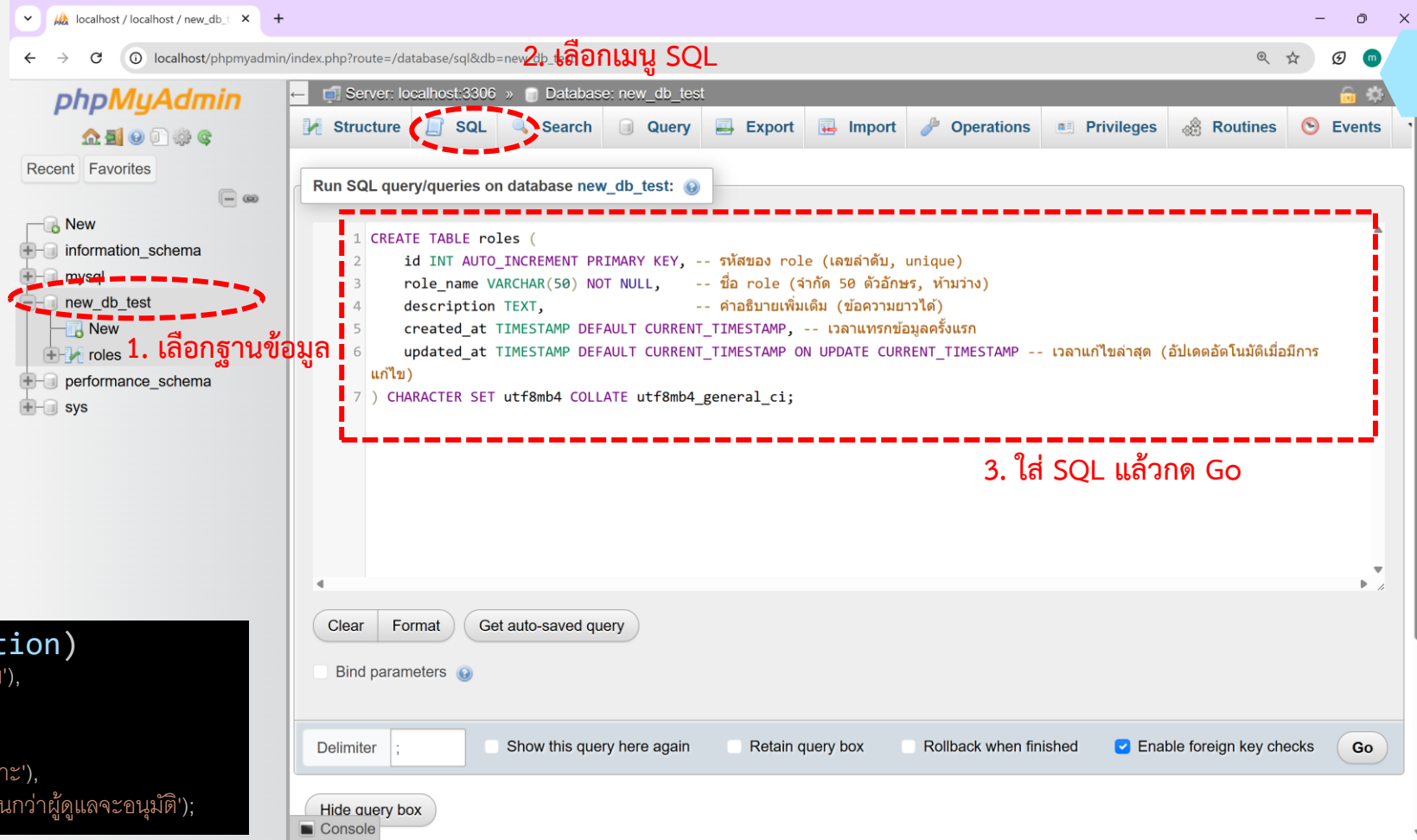
```
import express from 'express'
import dotenv from 'dotenv'
import IndexMiddleware from './src/index.middleware.js'
import IndexConfig from './src/index.config.js'
const app = express()
app.use(IndexConfig)
app.use(IndexMiddleware)
dotenv.config();

app.get('/', (req, res) => {
  res.status(200).send("Hello test!!!")
})

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`http://localhost:${PORT}`)
})
```

สร้างตาราง roles ที่ XAMPP โดยใช้คำสั่ง SQL



1. เลือกฐานข้อมูล

2. เลือกเมนู SQL

3. ใส่ SQL แล้วกด Go

ตัวอย่างการใส่ข้อมูล roles

```
INSERT INTO roles (role_name, description)
VALUES('Admin', 'ผู้ดูแลระบบ มีสิทธิ์จัดการทุกส่วนของระบบ'),
('Editor', 'ผู้แก้ไขเนื้อหาในระบบ แต่ไม่มีสิทธิ์จัดการผู้ใช้งาน'),
('Viewer', 'ผู้ชมเนื้อหาในระบบเท่านั้น ไม่สามารถแก้ไขข้อมูลได้'),
('Moderator', 'ผู้ดูแลคอมเมนต์และควบคุมการใช้งานฟีดเฉพาะ'),
('Contributor', 'ผู้มีสิทธิ์เพิ่มเนื้อหา แต่ไม่สามารถเผยแพร่ได้จนกว่าผู้ดูแลจะอนุมัติ);
```

สร้างตาราง roles

```
CREATE TABLE roles (
  id INT AUTO_INCREMENT PRIMARY KEY, -- รหัสของ role
  role_name VARCHAR(50) NOT NULL, -- ชื่อของ role
  description TEXT, -- คำอธิบายเพิ่มเติม
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, -- เวลาที่สร้างข้อมูล
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP -- เวลาที่แก้ไขล่าสุด
) CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
```

สร้างตาราง users ที่ XAMPP โดยใช้คำสั่ง SQL

สร้างตาราง users

```
CREATE TABLE users (
  id INT(3) AUTO_INCREMENT PRIMARY KEY, -- รหัสผู้ใช้งาน
  u_name VARCHAR(50) NOT NULL, -- ชื่อผู้ใช้งาน
  u_birth_date DATE NOT NULL, -- วันเกิดผู้ใช้งาน
  email VARCHAR(100), -- อีเมล
  phone_number VARCHAR(15), -- หมายเลขโทรศัพท์
  address TEXT, -- ที่อยู่
  salary DECIMAL(10, 2), -- เงินเดือน (ตัวอย่าง: 12345.67)
  is_active BOOLEAN DEFAULT 1, -- สถานะการใช้งาน (1 = active, 0 = inactive)
  role_id INT, -- FK ที่ชี้ไปยังตาราง roles
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, -- เวลาที่สร้างข้อมูล
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, -- เวลาที่แก้ไขล่าสุด
  FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE CASCADE ON UPDATE CASCADE
) CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
```

คำอธิบาย Foreign Key:

- **CONSTRAINT fk_role:** กำหนดชื่อข้อจำกัดว่า fk_role (Foreign Key ชื่อนี้ใช้ในการอ้างอิงหรือแก้ไขในอนาคต)
- **FOREIGN KEY (role_id):** ระบุว่า role_id ในตาราง users เป็น Foreign Key
- **REFERENCES roles(id):** ชี้ให้คอลัมน์ role_id ในตาราง users อ้างอิงถึงคอลัมน์ id ในตาราง roles
- **ON DELETE CASCADE:** หากข้อมูลในตาราง roles ที่ถูกอ้างอิงถูกลบ ข้อมูลใน users ที่เกี่ยวข้องจะถูกลบด้วย
- **ON UPDATE CASCADE:** หากค่า id ในตาราง roles ถูกอัปเดต ค่า role_id ในตาราง users จะถูกอัปเดตตาม

ตัวอย่างการใส่ข้อมูล users

```
-- ใส่ข้อมูลครบทุกคอลัมน์
INSERT INTO users (u_name, u_birth_date, email, phone_number, address, salary, is_active, role_id)
VALUES
('John Doe', '1990-01-15', 'john.doe@example.com', '0812345678', '123 Main St, Bangkok', 50000.00, 1, 1),
('Jane Smith', '1985-07-20', 'jane.smith@example.com', '0912345678', '456 Elm St, Chiang Mai', 45000.00, 1, 2),
('Emily Johnson', '1993-03-10', 'emily.johnson@example.com', '0898765432', '789 Pine St, Phuket', 55000.00, 0, 3);

-- ใส่ข้อมูลบางคอลัมน์
INSERT INTO users (u_name, u_birth_date, salary)
VALUES
('Michael Brown', '1992-12-25', 60000.00),
('Sarah Wilson', '1988-06-05', 47000.00);
```

ดูความสัมพันธ์ของตาราง

localhost / 127.0.0.1 / new_db_t x +

localhost/phpmyadmin/index.php?route=/database/designer&db=new_db_test

phpMyAdmin

Server: 127.0.0.1 » Database: new_db_test

Structure SQL Search Query Export Import Operations Privileges Routines Events Triggers **Designer**

Untitled *

Recent Favorites

New

- information_schema
- myfinal3
- mysql
- new_db_test**
- New
 - roles
 - users
- performance_schema
- phpmyadmin
- test
- test1

1. เลือกฐานข้อมูล

2. เลือก Designer

new_db_test roles

- id : int(11)
- role_name : varchar(50)
- description : text
- created_at : timestamp
- updated_at : timestamp

new_db_test users

- id : int(3)
- u_name : varchar(50)
- u_birth_date : date
- email : varchar(100)
- phone_number : varchar(15)
- address : text
- salary : decimal(10,2)
- is_active : tinyint(1)
- role_id : int(11)
- created_at : timestamp
- updated_at : timestamp

สร้างไฟล์ user.schema.js ในโฟลเดอร์ user

27

- ใน Node.js สามารถใช้ schema แบบนี้เพื่อเตรียมและ validate ข้อมูลก่อนส่งไปยัง SQL โดยการตรวจสอบว่าค่าต่าง ๆ ที่ได้รับจาก request นั้นตรงกับ schema ที่กำหนดไว้หรือไม่

user.schema.js

```
export const UserModels = {
  id: { type: 'INT', autoIncrement: true, primaryKey: true, required: false}, // 'ไม่จำเป็นต้องกำหนดค่า เพราะเป็น AUTO_INCREMENT
  u_name: { type: 'VARCHAR', length: 50, required: true}, // จำเป็นต้องมีค่า
  u_birth_date: { type: 'DATE', required: true}, // จำเป็นต้องมีค่า
  email: { type: 'VARCHAR', length: 100, required: false}, // 'ไม่จำเป็นต้องมีค่า
  phone_number: { type: 'VARCHAR', length: 15, required: false}, // 'ไม่จำเป็นต้องมีค่า
  address: { type: 'TEXT', required: false}, // 'ไม่จำเป็นต้องมีค่า
  salary: { type: 'DECIMAL', precision: 10, scale: 2, required: false}, // 'ไม่จำเป็นต้องมีค่า
  is_active: { type: 'BOOLEAN', defaultValue: 1, required: false}, // ค่าเริ่มต้นเป็น 1
  role_id: {
    type: 'INT',
    required: false, // 'ไม่จำเป็นต้องมีค่า
    foreignKey: {
      table: 'roles', // ชื่อตารางที่อ้างอิง
      column: 'id', // คอลัมน์ที่อ้างอิง
      onDelete: 'CASCADE', // ลบข้อมูลที่เกี่ยวข้องด้วย
      onUpdate: 'CASCADE' // อัปเดตข้อมูลที่เกี่ยวข้อง
    }
  },
  created_at: { type: 'TIMESTAMP', defaultValue: 'CURRENT_TIMESTAMP', required: false}, // ค่าเริ่มต้นเป็น CURRENT_TIMESTAMP
  updated_at: { type: 'TIMESTAMP', defaultValue: 'CURRENT_TIMESTAMP', onUpdate: 'CURRENT_TIMESTAMP', required: false }
// ค่าเริ่มต้นเป็น CURRENT_TIMESTAMP และอัปเดตอัตโนมัติ
};

export const TableName = "users"; // ชื่อตารางในฐานข้อมูล
```

ปกติใน MySQL เวลาเราเขียน INSERT มี 2 วิธีที่ใช้อยู่ ๆ

- วิธีที่ 1: ใช้ VALUES (มาตรฐาน SQL)

```
INSERT INTO users (username, email, password) VALUES ('john', 'john@test.com', '123456')
```

ชื่อตาราง

ชื่อคอลัมน์

ที่กำหนดให้แต่ละคอลัมน์

- ใช้กับ doQuery

```
const payload = ['alice', 'alice@test.com', 'hashed_pw'];
await doQuery(`INSERT INTO users (username, email, password) VALUES (?, ?, ?)` , payload);
```

- วิธีที่ 2: ใช้ SET (ฟีเจอร์ของ MySQL)

ชื่อตาราง

ชื่อคอลัมน์=ค่า

```
INSERT INTO users SET username='john', email='john@test.com', password='123456';
```

- วิธีการที่ 2 ผลลัพธ์ เหมือนกัน กับวิธีแรก (ต่างกันแค่ style) วิธีการที่ 2 เป็น syntax ที่ MySQL เพิ่มมาเอง (SQL มาตรฐานไม่มี) ข้อดีคือเวลาใช้กับ Node.js driver (mysql2/mysql) สามารถทำแบบนี้ได้:

- ใช้กับ doQuery

```
const payload = { username: "john", email: "john@test.com", password: "123456" };
await doQuery("INSERT INTO users SET ?", payload);
```

- Driver จะ map object ให้เป็น field=value โดยอัตโนมัติ

เปรียบเทียบใช้ VALUES และ SET

แบบ	Syntax	ใช้กับ Node.js Object
VALUES	INSERT INTO users (col1, col2) VALUES (?, ?)	ใช้กับ array → ["john", "john@test.com"]
SET	INSERT INTO users SET col1=?, col2=?	ใช้กับ object → { username: "john", email: "john@test.com" }

- SET ? เหมาะกับ object → เขียนสั้น & ปลอดภัย (driver ช่วย escape ให้)
- VALUES (?, ?, ?) เหมาะกับ array → ชัดเจนเวลา insert หลายคอลัมน์ตามลำดับ

ไฟล์ user.service.js

- doQuery: ฟังก์ชันที่นำเข้ามาจาก mysql.db.js ซึ่งใช้สำหรับการดำเนินการคำสั่ง SQL บนฐานข้อมูล UserModels, TableName: ข้อมูลที่นำเข้ามาจาก user.schema.js ซึ่งประกอบด้วย:
 - TableName: ชื่อตารางในฐานข้อมูลที่ใช้สำหรับการดำเนินการ (ในที่นี้คือ "users")
 - UserModels: การกำหนดรูปแบบของข้อมูลซึ่งมาจากไฟล์ ไฟล์ user.schema.js

user.service.js

```
import { doQuery } from "../mysql.db.js";
import { UserModels, TableName } from './user.schema.js'
const UserService = {
  createUser: (payload) => {
    return doQuery(`INSERT INTO ${TableName} SET ?`, payload)
  }
}
export default UserService
```

- คำสั่ง doQuery(INSERT INTO \${TableName} SET ?, payload) เป็นการใช้นำคำสั่ง **SQL INSERT** แบบ dynamic เพื่อเพิ่มข้อมูลลงในตาราง โดยใช้รูปแบบ **parameterized query** ซึ่งช่วยป้องกัน **SQL injection** และทำให้โค้ดดูสะอาดและยืดหยุ่นมากขึ้น. (SQL Injection เป็นการโจมตีทางเว็บที่แฮกเกอร์ใช้เพื่อดึงข้อมูลหรือแก้ไขข้อมูลในฐานข้อมูลผ่านช่องโหว่ของระบบที่มีการใช้คำสั่ง SQL ที่ไม่ปลอดภัย โดยแฮกเกอร์สามารถใส่โค้ด SQL ที่เป็นอันตรายลงไปในเว็บไซต์ ฟอรัม ล็อกอิน หรือ URL เพื่อให้ระบบรันคำสั่ง SQL ที่แฮกเกอร์ต้องการได้)
 - INSERT INTO \${TableName}: ใช้เพื่อระบุว่าคุณต้องการเพิ่มข้อมูลเข้าไปในตารางที่ชื่อตารางถูกแทนด้วยค่าของตัวแปร TableName (เช่น 'users').
 - SET ?: คำสั่ง SQL ถูกใช้เพื่อกำหนดค่าของคอลัมน์ต่าง ๆ ในตารางฐานข้อมูล โดย ? เป็น placeholder สำหรับค่าที่จะถูกแทน จะถูกแปลงเป็น key-value pair ในรูปแบบ SQL

สร้างไฟล์ user.controller.js

- UserController เป็นอ็อบเจกต์ที่มีเมธอด addUser
- addUser ซึ่งทำหน้าที่จัดการกับการร้องขอ POST เพื่อเพิ่ม users ใหม่
- typeof is_active === 'boolean'
 - ตรวจสอบว่าตัวแปร is_active ที่รับมาเป็นชนิดข้อมูล boolean (true หรือ false) หรือไม่?
is_active : true
 - ถ้าเป็น boolean จริง → ใช้ค่าที่ส่งมา (is_active)
 - ถ้าไม่ใช่ boolean → กำหนดค่าเป็น true โดยอัตโนมัติ

user.controller.js

31

```
import UserService from '../services/user.service.js';
const UserController = {
  addUser: async (req, res) => {
    try {
      const {u_name,u_birth_date,email,phone_number,address,salary,is_active,role_id} = req.body;
      // ตรวจสอบว่า 'u_name' และ 'u_birth_date' เป็นข้อมูลที่จำเป็น
      if (!u_name || !u_birth_date) {
        return res.status(400).json({
          success: false,
          message: 'u_name และ u_birth_date เป็นข้อมูลที่จำเป็น'
        });
      }

      const created = await UserService.createUser({
        u_name,
        u_birth_date,
        email: email || null, // ค่า default เป็น null ถ้าไม่มีใน body
        phone_number: phone_number || null,
        address: address || null,
        salary: salary || null,
        is_active: typeof is_active === 'boolean' ? is_active : true, // ค่า default เป็น true
        role_id: role_id || null
      });

      res.status(201).json({
        success: true,
        data: created,
        URL: req.url
      });
    } catch (error) {
      console.error('Error in addUser:', error);
      res.status(500).json({
        success: false,
        message: 'เกิดข้อผิดพลาดในระบบ',
        error: error.message
      });
    }
  }
};
export default UserController;
```

ไฟล์ create-user.dto.js

- ติดตั้ง Joi สำหรับการตรวจสอบข้อมูลที่ได้รับมาจากการร้องขอ (requests) หรือข้อมูลที่ใช้กรอกเข้ามาเพื่อให้แน่ใจว่าข้อมูลมีรูปแบบและค่าที่ถูกต้องตามที่ต้องการ
- ติดตั้ง createValidator ใช้สร้างตัวตรวจสอบ (validator) ที่สามารถใช้ในการตรวจสอบข้อมูลที่ได้รับจาก request ผ่านคำสั่ง:
- `npm install joi express-joi-validation`

จุดเด่นของ schema นี้

1. ป้องกันข้อมูลไม่ถูกต้อง ก่อนถึง DB เช่น

1. ถ้าใส่ phone_number="abc123" → ไม่ผ่าน เพราะไม่ match regex
2. ถ้าใส่ salary=-100 → ไม่ผ่าน เพราะ positive()

2. ลดปัญหา SQL Error

→ validate ที่ฝั่ง Node.js ก่อน insert/update

3. ค่า default

1. is_active ถ้าไม่ได้ส่งมา → จะได้ true อัตโนมัติ

create-user.dto.js

32

```
import Joi from 'joi';

const CreateUserDto = Joi.object({
  u_name: Joi.string().max(50).required(),
  // ชื่อผู้ใช้ ต้องเป็น string, ความยาวไม่เกิน 50 ตัวอักษร, ห้ามเว้นว่าง (NOT NULL)
  u_birth_date: Joi.date().iso().required(),
  // วันเกิด ต้องเป็นชนิด date, รูปแบบ ISO (YYYY-MM-DD), ห้ามว่าง
  email: Joi.string().email().max(100).optional(),
  // อีเมล ต้องเป็น string รูปแบบ email, ความยาวไม่เกิน 100 ตัวอักษร, ใส่หรือไม่ใส่ก็ได้
  phone_number: Joi.string()
    .pattern(/^([0-9]{10})$/)
    .optional(),
  // เบอร์โทรศัพท์ ต้องเป็น string มีแค่ตัวเลข 10 หลัก, ใส่หรือไม่ใส่ก็ได้
  address: Joi.string().max(65535).optional(),
  // ที่อยู่ เป็น string ความยาวไม่เกิน 65535 ตัวอักษร (TEXT), ไม่บังคับ
  salary: Joi.number().precision(2).positive().optional(),
  // เงินเดือน เป็น number, ทศนิยม 2 ตำแหน่ง, ต้องเป็นบวก, ไม่บังคับ
  is_active: Joi.boolean().default(true),
  // สถานะการใช้งาน เป็น boolean, ถ้าไม่ส่งมา → default = true
  role_id: Joi.number().integer().optional(),
  // foreign key ขึ้นที่ role_id, เป็น integer, ไม่บังคับ
});

export default CreateUserDto;
```

Regular Expression (Regex)

```
.pattern(/^[0-9]{10}$/)
```

ความหมายการใช้ Regular Expression (Regex)

- ^ → บอกว่าเริ่มต้น string ต้องเป็นไปตามเงื่อนไขนี้
- [0-9] → อนุญาตเฉพาะตัวเลข 0 ถึง 9
- {10} → ต้องมีจำนวน 10 หลักพอดี
- \$ → บอกว่าสิ้นสุด string ต้องตรงนี้ (ไม่ยาวไปกว่านี้)

ความต้องการ	Regex	ตัวอย่าง ผ่าน	ตัวอย่าง ไม่ผ่าน
เบอร์โทร 10 หลัก (ไทย)	/^[0-9]{10}\$/	0812345678	08123, 08123abc
รหัสไปรษณีย์ไทย 5 หลัก	/^[0-9]{5}\$/	30000	123, ABCDE
รหัสผ่าน ≥8 ตัว (มีพิมพ์เล็ก-ใหญ่-ตัวเลข-พิเศษ)	/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@\$!%*?&])[A-Za-z\d@\$!%*?&]{8,}\$/	Abc123!!	abc123, ABCDEF
Username (a-z, 0-9, _, 3-20 ตัว)	/^[a-zA-Z0-9_]{3,20}\$/	user_01, Alice99	ab, @user!
อีเมล	/^[\\w._%+-]+@[A-Za-z0-9.-]+\\. [A-Za-z]{2,}\$/	test@mail.com	test@mail, @mail.com
URL (http/https)	/^(https?:\\V)?(\\w- \\.[\\w-]+)(\\w.,@?^=%&:/~+#+-]*)?\$/	https://example.com, http://abc.org/test?id=1	ftp://test.com, example
เฉพาะภาษาไทย	/^[ก-๙\\s]+\$/	สมชาย ใจดี	Somchai, 123
เฉพาะภาษาอังกฤษ (A-Z, a-z)	/^[A-Za-z]+\$/	HelloWorld	Hello123, สวัสดี
ตัวเลขเท่านั้น	/^[0-9]+\$/	123456	12a3, abc
รหัสนักศึกษา (2 ตัวอักษร + 6 ตัวเลข)	/^[A-Za-z]{2}[0-9]{6}\$/ หรือ /^[AB][0-9]{6}\$/ บังคับให้ขึ้นต้นด้วย AB เท่านั้น	CE650001, IT123456	C650001, 12345678
เลขบัตร ปชช. ไทย (13 หลัก)	/^[0-9]{13}\$/	1234567890123	12345, abc1234567890
เบอร์โทรสากล (E.164)	/^[\\+][1-9][0-9]{8,14}\$/	+66812345678, +14155552671	0812345678, ++661234

ไฟล์ user.route.js ที่โฟลเดอร์ users

- เป็นไฟล์ที่ใช้กำหนด route ในการเข้าถึง user
- ใช้ validator เป็น middleware ใน route โดยใช้ไฟล์ในโฟลเดอร์ dto ซึ่งตอนนี้คือ create-user.dto.js

user.route.js

```
import express from 'express';
import UserController from './user.controller.js';
import { createValidator } from 'express-joi-validation';
import CreateUserDto from './create-user.dto.js';
const UserRouter = express.Router();
const validator = createValidator();
UserRouter.post('/', validator.body(CreateUserDto),
UserController.addUser);
export default UserRouter;
```

ไฟล์ index.route.js ที่ไฟล์เดอร์ src

- IndexRouter.use('/user', UserRouterRouter): ใช้ UserRouterRouter เป็น router ที่จัดการการร้องขอที่เริ่มต้นด้วย /user
- ทุกเส้นทางที่เริ่มต้นด้วย /user จะถูกส่งไปยัง UserRouterRouter ซึ่งมีการกำหนดเส้นทางที่เกี่ยวข้องกับการจัดการผู้ใช้ เช่น การสร้างผู้ใช้, การดึงข้อมูลผู้ใช้ ฯลฯ

index.route.js

```
import express from 'express'
import UserRouterRouter from './users/user.route.js'
const IndexRouter = express();
IndexRouter.use('/user', UserRouterRouter)
export default IndexRouter
```

เพิ่มการเรียกใช้ไฟล์ user.route.js ลงใน ไฟล์ index.route.js

ไฟล์ index.js

- เพิ่มการเรียกใช้ไฟล์ index.route.js ลงใน ไฟล์ index.js

```
import express from 'express'
import dotenv from 'dotenv'
import IndexMiddleware from './src/index.middleware.js'
import IndexConfig from './src/index.config.js'
import IndexRouter from './src/index.route.js'
dotenv.config();
const app = express()
app.use(IndexConfig)
app.use(IndexMiddleware)
app.use(IndexRouter)

app.get('/', (req, res) => {
  res.status(200).send("Hello test!!!")
})

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`http://localhost:${PORT}`)
})
```

index.js

ทดลองใช้ postman ส่ง request โดย POST
จะต้องมีข้อมูลเพิ่มใน Laragon ที่ตาราง users

Home Workspaces API Network

POST http://localhost:3000/user

POST http://localhost:3000/user

Params Authorization Headers (8) Body Scripts

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw

```
1 {  
2   "u_name": "John Doe",  
3   "u_birth_date": "1990-01-01",  
4   "email": "john.doe@example.com",  
5   "phone_number": "0812345678",  
6   "address": "123 Main Street, Bangkok",  
7   "salary": 50000.00,  
8   "is_active": true,  
9   "role_id": 1  
10 }
```

localhost / 127.0.0.1 / new_db_test

localhost/phpmyadmin/index.php?route=/sql&pos=0&db=new_db_test&table=users

Server: 127.0.0.1 Database: new_db_test Table: users

Showing rows 0 - 5 (6 total, Query took 0.0003 seconds.)

SELECT * FROM `users`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all Number of rows: 25 Filter rows: Search this table Sort by key: None

	id	u_name	u_birth_date	email	phone_number	a
☐	1	John Doe	1990-01-15	john.doe@example.com	0812345678	1: B
☐	2	Jane Smith	1985-07-20	jane.smith@example.com	0912345678	4: M
☐	3	Emily Johnson	1993-03-10	emily.johnson@example.com	0898765432	7: F
☐	4	Michael Brown	1992-12-25	NULL	NULL	N
☐	5	Sarah Wilson	1988-06-05	NULL	NULL	N
☐	6	John Doe	1990-01-01	john.doe@example.com	0812345678	1: B

ทำส่วนดึงข้อมูล user ทั้งหมดมาแสดง ไฟล์ user.service.js

39

ไฟล์ user.service.js

```
import { UserModels, TableName } from '../model/user.schema.js'
const UserService = {
  createUser: (payload) =>{
    return doQuery(`INSERT INTO ${TableName} SET ?`,payload)
  },
  getUserAll: (columns = Object.keys(UserModels)) => {
    const options = [columns, TableName];
    return doQuery(`SELECT ?? FROM ??`, options);
  }
}
export default UserService
```

?: ใช้สำหรับการแทนค่าของ ข้อมูล (values) ในคำสั่ง SQL

?: ใช้สำหรับการแทน ชื่อคอลัมน์หรือชื่อของตาราง (identifiers) ในคำสั่ง SQL

- columns = Object.keys(UserModels)
 - ถ้าไม่ได้ส่ง argument columns เข้ามา → จะใช้ชื่อฟิลด์ทั้งหมดที่อยู่ใน UserModels (เช่น { id: ..., name: ..., email: ... } → จะได้ ['id','name','email'])
- options = [columns, TableName]
 - columns = รายชื่อคอลัมน์ที่จะ select
 - TableName = ชื่อตาราง (เช่น "users")
 - ทั้งหมดถูกส่งเป็น array เพื่อให้ MySQL driver ไปแทนค่า ??
- SELECT ?? FROM ??
 - ใน MySQL driver (mysql2 หรือ mysql)
 - ?? = placeholder สำหรับ **identifier** เช่น ชื่อ column หรือชื่อ table
 - ต่างจาก ? ที่ใช้กับ values
 - เช่น: doQuery("SELECT ?? FROM ??", [{"id", "name"}, "users"])
 - จะกลายเป็น SELECT `id`, `name` FROM `users`;
- return doQuery(...)
 - ฟังก์ชันนี้ส่ง Promise กลับ → เวลาจะใช้ต้อง await UserService.getUserAll()

ทำส่วนดึงข้อมูล user ทั้งหมดมาแสดง ไฟล์ user.controller.js

ไฟล์ user.controller.js

```
import UserService from '../services/user.service.js'
const UserController = {
  addUser:
    ...
}
,showAllUser: async (req,res)=>{
  try {
    const users = await UserService.getUserAll();
    res.status(200).json({
      success: true,
      data: users,
      URL: req.url
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: error.message
    });
  }
}
}
export default UserController
```

ทำส่วนดึงข้อมูล user ทั้งหมดมาแสดง ไฟล์ user.route.js

ไฟล์ user.route.js

```
import express from 'express';
import UserController from './user.controller.js';
import { createValidator } from 'express-joi-validation';
import CreateUserDto from './create-user.dto.js';
const UserRouter = express.Router();
const validator = createValidator();
UserRouter.post('/', validator.body(CreateUserDto),
UserController.addUser)
UserRouter.get('/',UserController.showAllUser)
export default UserRouter;
```

ทดลองใช้ postman ส่ง request โดย GET จะต้องมียข้อมูล users แสดงกลับมา

The screenshot displays the Postman interface with a GET request configured to `http://localhost:3000/user`. The request is sent, resulting in a `200 OK` status. The response body is shown in JSON format, containing a success flag and a list of user data.

Request Details:

- Method: GET
- URL: `http://localhost:3000/user`
- Body: none

Response Details:

- Status: 200 OK
- Time: 33 ms
- Size: 1.91 KB

Response Body (JSON):

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "u_name": "John Doe",
      "u_birth_date": "1990-01-14T17:00:00.000Z",
      "email": "john.doe@example.com",
      "phone_number": "0812345678",
      "address": "123 Main St, Bangkok",
      "salary": 50000,
      "is_active": 1,
      "role_id": 1,
      "created at": "2025-02-13T10:10:21.000Z"
    }
  ]
}
```

The interface also shows tabs for Params, Authorization, Headers (6), Body, Scripts, and Settings. The Body tab is currently selected, showing the JSON response. The bottom status bar includes options like Postbot, Runner, Start Proxy, Cookies, Vault, Trash, and a help icon.

- ไปที่ ไฟล์ user.service.js เพิ่มโค้ด

ถ้าต้องการดึงข้อมูล user ด้วย id

```
,getUserById:(id = {})=>{
  const sql = `SELECT * FROM ${TableName} WHERE id = ?`; // ใช้ ? แทนที่ค่าจริง
  const options = [id]; // ค่าแทนที่ ? คือ id
  return doQuery(sql, options);
}
```

- ไปที่ ไฟล์ user.controller.js เพิ่มโค้ด

```
,getUserById : async (req,res)=>{
  const {id} = req.params
  const users = await UserService.getUserById(id)
  res.status(200).json({
    success:true,
    data: users
  })
}
```

- ไปที่ ไฟล์ user.route.js เพิ่มโค้ด

```
UserRouter.get('/:id', UserController.getUserById)
```

```
const found = await doQuery(
  `SELECT *FROM ${TableName} WHERE id LIKE ?`, ['%1%']
);
```

ทดลองใช้ postman ส่ง request โดย GET ส่งค่า :id = 3 จะต้องมีข้อมูล users ที่ id =3 แสดงกลับมา

The screenshot shows the Postman interface with a GET request to `http://localhost:3000/user/3`. The request is successful, returning a 200 OK status with a response time of 6 ms and a body size of 587 B. The response body is a JSON object containing user information for id 3.

Request:

- Method: GET
- URL: `http://localhost:3000/user/3`
- Body: none

Response:

```
3  "data": [  
4    {  
5      "id": 3,  
6      "u_name": "Emily Johnson",  
7      "u_birth_date": "1993-03-09T17:00:00.000Z",  
8      "email": "emily.johnson@example.com",  
9      "phone_number": "0898765432",  
10     "address": "789 Pine St, Phuket",  
11     "salary": 55000,  
12     "is_active": 0,  
13     "role_id": 3,  
14     "created_at": "2025-02-13T10:10:21.000Z",  
15     "updated_at": "2025-02-13T10:10:21.000Z"  
16   }  
]
```

ถ้าต้องการอัปเดตข้อมูล users

- ไปที่ ไฟล์ user.service.js เพิ่มโค้ด
- const allowed = new Set([...]);
 - กำหนดคอลัมน์ที่อนุญาตให้แก้ไข
 - ป้องกันไม่ให้ผู้ใช้ส่ง key แปลก ๆ มาอัปเดต
- สร้าง object ว่า safePayload เพื่อเก็บเฉพาะคีย์ที่ตรวจสอบแล้วว่าอนุญาต
- Object.entries(payload) แปลง object → array ของคู่ [key, value]
 - เช่น const payload = { u_name: "John", email: "john@test.com" };
 - Object.entries(payload);
 - จะได้ [["u_name", "John"], ["email", "john@test.com"]]
- if (allowed.has(k))
 - ตรวจว่า key (k) นี้อยู่ใน allowed (Set ของคอลัมน์ที่อนุญาตให้ update) หรือไม่
- safePayload[k] = v;
 - ถ้าผ่านเงื่อนไข → เก็บค่าไว้ใน object safePayload

ไฟล์ user.service.js

45

```
,updateOneUserById : (id, payload) => {
    const allowed = new Set([
        'u_name','u_birth_date','email','phone_number',
        'address','salary','is_active','role_id'
    ]);
    const safePayload = {};

    for (const [k, v] of Object.entries(payload)) {
        if (allowed.has(k)) safePayload[k] = v;
    }

    const sql = 'UPDATE ?? SET ? WHERE id = ?';
    const params = [TableName, safePayload, id];
    return doQuery(sql, params);
}
```

ถ้าต้องการอัปเดตข้อมูล users (ต่อ)

- ไปที่ ไฟล์
user.controller.js
เพิ่มโค้ด
- Object.prototype.hasOwnProperty.call(...)
เป็นวิธีที่ปลอดภัยที่สุด
ในการเช็ค ว่า object มี
property นั้นจริง ๆ
(โดยตรงจากตัว object
เอง ไม่ใช่สืบทอดมา
จาก prototype)
- affectedRows =
จำนวนแถว (rows) ใน
ตารางที่ได้รับผลกระทบ
จากคำสั่ง SQL
 - ถ้าค่าใหม่ที่อัปเดต
เหมือนกับค่าเดิมเป๊ะ
affectedRows จะ
ยังเป็น 1 แต่
changedRows จะ
เป็น 0

ไฟล์ user.controller.js

```
,updateUser: async (req, res) => {
  try {
    const { id } = req.params;
    if (!id) return res.status(400).json({ success: false, message: 'ID is required' });

    const {u_name,u_birth_date,email,phone_number,address,salary, is_active,role_id} = req.body;

    const payload = {};
    if (Object.prototype.hasOwnProperty.call(req.body, 'u_name')) payload.u_name = u_name;
    if (Object.prototype.hasOwnProperty.call(req.body, 'u_birth_date')) payload.u_birth_date = u_birth_date;
    if (Object.prototype.hasOwnProperty.call(req.body, 'email')) payload.email = email;
    if (Object.prototype.hasOwnProperty.call(req.body, 'phone_number')) payload.phone_number = phone_number;
    if (Object.prototype.hasOwnProperty.call(req.body, 'address')) payload.address = address;
    if (Object.prototype.hasOwnProperty.call(req.body, 'salary')) payload.salary = salary;
    if (Object.prototype.hasOwnProperty.call(req.body, 'is_active')) payload.is_active = is_active;
    if (Object.prototype.hasOwnProperty.call(req.body, 'role_id')) payload.role_id = role_id;

    if (Object.keys(payload).length === 0) {
      return res.status(400).json({ success: false, message: 'No fields to update' });
    }

    const result = await UserService.updateOneUserById(id, payload);

    if (result.affectedRows === 0) {
      return res.status(404).json({ success: false, message: 'User not found' });
    }
    return res.status(200).json({
      success: true,
      affectedRows: result.affectedRows,
      changedRows: result.changedRows,
    });
  } catch (err) {
    console.error(`Error updating user: ${err.message}`);
    return res.status(500).json({ success: false, message: `Error updating user: ${err.message}` });
  }
}
```

ถ้าต้องการอัปเดตข้อมูล users (ต่อ)

- สร้างไฟล์ update-user.dto.js ในโฟลเดอร์ dto

ไฟล์ update-user.dto.js

```
import Joi from 'joi';

const UpdateUserDto = Joi.object({
  u_name: Joi.string().max(50).optional(), // ชื่อผู้ใช้งาน (VARCHAR(50), ไม่บังคับ)
  u_birth_date: Joi.date().iso().optional(), // วันเกิดในรูปแบบ ISO 8601 (YYYY-MM-DD), ไม่บังคับ
  email: Joi.string().email().max(100).optional(), // อีเมลที่ถูกต้อง (VARCHAR(100), ไม่บังคับ)
  phone_number: Joi.string()
    .pattern(/^([0-9]){10}$/) // หมายเลขโทรศัพท์ประเทศไทย ต้องเป็นตัวเลข 10 หลัก
    .message('Phone number must be 10 digits and numeric.') // ข้อความ error ที่ชัดเจน
    .optional(),
  address: Joi.string().max(65535).optional(), // ที่อยู่ (TEXT, ไม่บังคับ)
  salary: Joi.number()
    .precision(2) // เงินเดือน รองรับตัวเลขทศนิยม 2 ตำแหน่ง
    .positive() // ต้องเป็นตัวเลขบวก
    .optional(),
  is_active: Joi.boolean().optional(), // สถานะการใช้งาน (BOOLEAN, ไม่บังคับ)
  role_id: Joi.number().integer().optional(), // รหัสบทบาท (INT, ไม่บังคับ)
});

export default UpdateUserDto;
```

ไม่ได้ระบุ id ใน schema เพราะ id ควรจะอยู่ใน URL params

- ไปที่ไฟล์ user.route.js เพิ่มโค้ด

ไฟล์ user.route.js

```
import UpdateUserDto from './update-user.dto.js';
UserRouter.patch('/:id', validator.body(UpdateUserDto), UserController.updateUser);
```

ทดลองใช้ postman ส่ง request โดย PATCH จะต้องมีการอัปเดตข้อมูล users อัปเดตเงินเดือนและสถานการณ์ใช้งาน ของ id = 1

48

ดึงข้อมูล ของ id = 1

API Network

GET http://localhost:3000/user/1

Params Authorization Headers (6) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary

Body Cookies Headers (8) Test Results

{ } JSON Preview Visualize

```
1 {
2   "success": true,
3   "data": [
4     {
5       "id": 1,
6       "u_name": "John Doe",
7       "u_birth_date": "1990-01-14T17:00:00.000Z",
8       "email": "john.doe@example.com",
9       "phone_number": "0812345678",
10      "address": "123 Main St, Bangkok",
11      "salary": 50000,
12      "is_active": 1,
13      "role_id": 1,
14      "created_at": "2025-02-13T10:10:21.000Z",
15      "updated_at": "2025-02-13T10:10:21.000Z"
16    }
17  ]
18 }
```

อัปเดตข้อมูล ของ id = 1

API Network

PUT http://localhost:3000/user/1

Params Authorization Headers (8) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary

Body Cookies Headers (8) Test Results

```
1 {
2   "salary": 75000.00,
3   "is_active": false
4 }
```

ดึงข้อมูล ของ id = 1 หลังอัปเดต

API Network

GET http://localhost:3000/user/1

Params Authorization Headers (8) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary

Body Cookies Headers (8) Test Results

{ } JSON Preview Visualize

```
1 {
2   "success": true,
3   "data": [
4     {
5       "id": 1,
6       "u_name": "John Doe",
7       "u_birth_date": "1990-01-14T17:00:00.000Z",
8       "email": "john.doe@example.com",
9       "phone_number": "0812345678",
10      "address": "123 Main St, Bangkok",
11      "salary": 75000,
12      "is_active": 0,
13      "role_id": 1,
14      "created_at": "2025-02-13T10:10:21.000Z",
15      "updated_at": "2025-02-13T13:42:09.000Z"
16    }
17  ]
18 }
```

ถ้าต้องการลบข้อมูล users

- ไปที่ ไฟล์ user.service.js เพิ่มโค้ด

```
,deleteUserId:(id = {})=>{  
  // สร้างคำสั่ง SQL สำหรับการลบข้อมูล  
  const sql = `DELETE FROM ${TableName} WHERE id = ?`;   
  const options = [id]  
  return doQuery(sql, options);  
}
```

ไฟล์ user.service.js

ถ้าต้องการลบข้อมูล users (ต่อ)

- ไปที่ ไฟล์ user.controller.js เพิ่มโค้ด

ไฟล์ user.controller.js

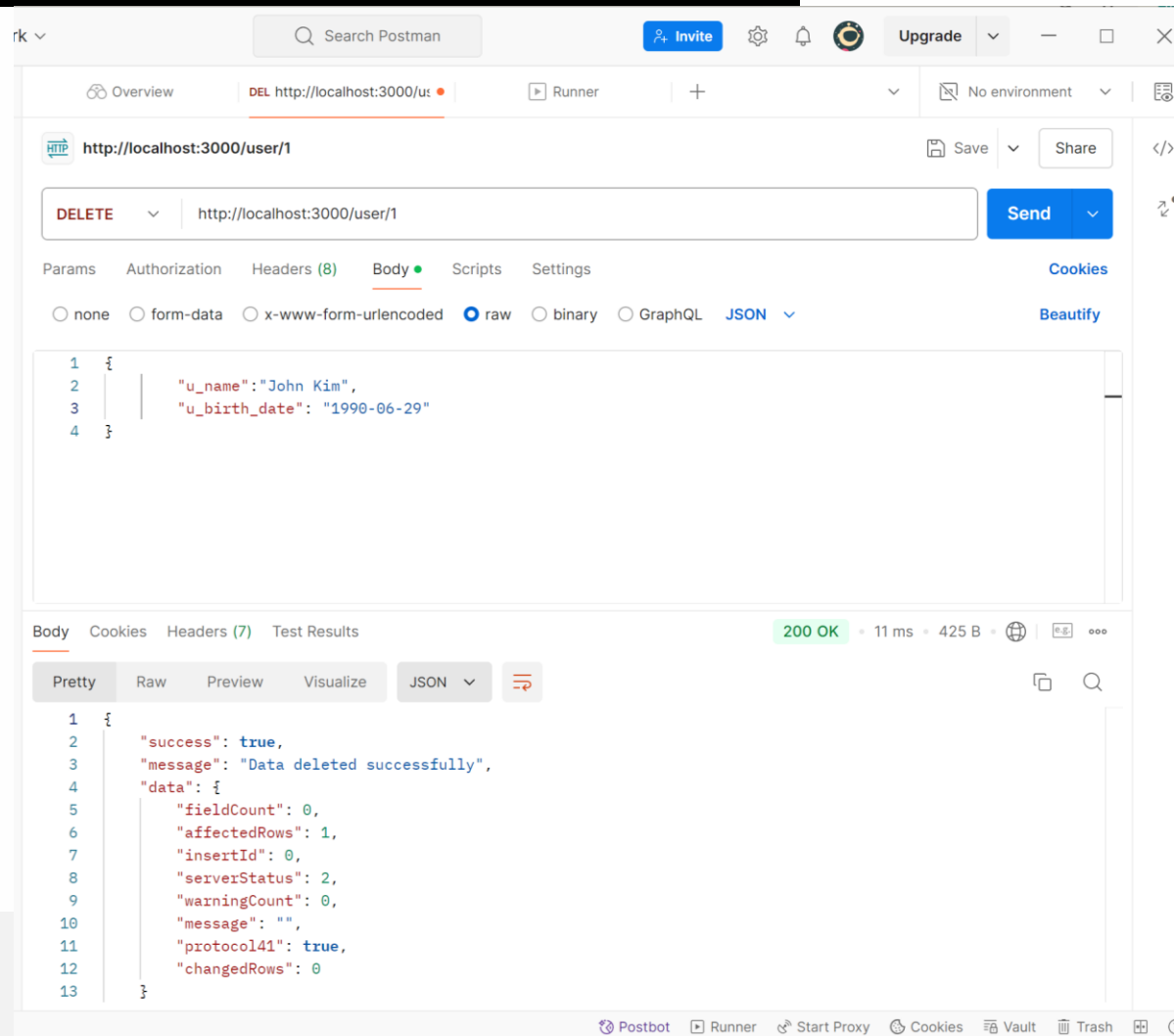
```
,deleteUser : async (req,res)=>{
  try {
    const { id } = req.params;
    if (!id) {
      return res.status(400).json({ success: false, message: 'ID is required' });
    }
    const result = await UserService.deleteUserById(id)
    // ตรวจสอบว่ามีแถวที่ถูกลบหรือไม่
    if (result.affectedRows === 0) {
      return res.status(404).json({ success: false, message: 'Data not found' });
    }
    res.status(200).json({
      success:true,
      message: 'Data deleted successfully',
      data: result
    })
  } catch (err) {
    console.error(`Error deleting data: ${err.message}`);
    res.status(500).json({ success: false, message: `Error deleting data: ${err.message}` });
  }
}
```

ถ้าต้องการลบข้อมูล users (ต่อ)

- ไปที่ ไฟล์ user.route.js เพิ่มโค้ด

ไฟล์ user.route.js

```
UserRouter.delete('/:id', UserController.deleteUser);
```



ใบงาน

- ให้ทดลองสร้าง route แบบ `.get('/users/search', ...)`
- request ส่งค่า query มาให้ นำค่า query ไปค้นหาใน `u_name` โดยจะอยู่ตำแหน่งใดใน `u_name` ก็ได้ ผลลัพธ์แสดงแค่ `id` และ `u_name`
- โดยสามารถดึงค่า query ได้จาก `req.query`
- ข้อควรระวังให้วาง route ก่อน `/:id` เช่น เมื่อยิง `GET /users/search?keyword=J` ตัว Router จะ แมตช์ `/search` เข้ากับ `/:id` (จับค่า `id = 'search'`) แล้วส่งไป `getUserById` แทน

```
UserRouter.get('/search', UserController.showSearch)
UserRouter.get('/:id', UserController.getUserById)
```

คำแนะนำสามารถใช้ LIKE ใน SQL เพื่อค้นหาข้อมูลตาม pattern (บางส่วนของข้อความ)

- ตัวอย่าง `SELECT * FROM users WHERE phone_number LIKE '%8';`
 - คือ แสดงทุกคอลัมน์ในตาราง `users` ที่ `phone_number` ลงท้ายด้วย 8
- การใช้ LIKE ใช้คู่กับ wildcard
 - `%` = แทนได้ทุกจำนวนตัวอักษร (รวมถึง 0 ตัว)
 - `_` = แทนได้ 1 ตัวอักษร
 - เช่น
 - ขึ้นต้นด้วย "Jo" ใช้ `LIKE 'Jo%'`
 - ลงท้ายด้วย "son" `LIKE '%son';`
 - มี "ohn" อยู่ตรงไหนก็ได้ในข้อความ `LIKE '%ohn%';`
 - -มีตัวอักษร 3 ตัว แล้วตามด้วย "hn" `LIKE '___hn';`

ตัวอย่างผลลัพธ์

GET ▼ http://localhost:3500/user/search?keyword=J

Params ● Authorization Headers (6) Body Scripts Settings

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ Gra

This request does not have a body

Body Cookies Headers (8) Test Results ↺

{} JSON ▼ ▶ Preview 🖼 Visualize ▼

```

1  {
2    "success": true,
3    "data": [
4      {
5        "id": 1,
6        "u_name": "JJ"
7      },
8      {
9        "id": 2,
10       "u_name": "Jane Smith"
11     },
12   ]
  
```

Console

GET ▼ http://localhost:3500/user/search?keyword=a

Params ● Authorization Headers (6) Body Scripts Settings

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ Grap

This request does not have a body

Body Cookies Headers (8) Test Results ↺

{} JSON ▼ ▶ Preview 🖼 Visualize ▼

```

1  {
2    "success": true,
3    "data": [
4      {
5        "id": 2,
6        "u_name": "Jane Smith"
7      },
8      {
9        "id": 4,
10       "u_name": "Michael Brown"
11     },
12   ]
  
```

Console